

第十一章

软件实现

本章要点

- 一、关于编码的一些理念
- 二、编程语言的选择
- 三、编码标准和规范
- 四、程序效率

引言

软件的详细设计完成，就表示完成了软件的过程性的描述，进入软件编码阶段。

编码（Coding）阶段的任务简单说，是为每个模块编写程序。即是将详细设计的结果转换为用某种计算机语言写的程序——源代码。

在软件生命期中，程序经常需要被人阅读和理解，如何提高程序的可读性（Readability）？使程序“简单”和“清晰”，进而使程序具有良好的可靠性、可维护性，这是非常重要的。

引言

本单元不是介绍如何编写程序，而是从如何提高软件的质量和可维护性的角度，讨论在编码阶段所要解决的主要问题：

- **程序设计语言的特性及选择的原则**
- **编码风格**

什么是软件编码？

Programming ? = Coding

- 一种观点

- 软件编码是将软件设计模型机械地转换成源程序代码，这是一种低水平的、缺乏创造性的工作。
- 软件程序员是所谓的“软件蓝领”。

- 问题：

- 你是否认同这种观点？
- 如果不认同，你如何看待软件编码？

什么是软件编码？

Professional Programmer = Software Engineer

- 正确观点

软件编码是一个复杂而迭代的过程，包括程序设计和程序实现。

- 软件编码要求

- 正确地理解用户需求和软件设计思想
- 正确地根据设计模型进行程序设计
- 正确而高效率地编写和测试源代码
- 软件编码是设计的继续，会影响软件质量和可维护性。
- 软件编码要考虑重用和重构



软件编码的工作

- 程序设计
 - 理解软件的需求说明和设计模型
 - 补充遗漏的或剩余的详细设计
 - 设计程序代码的结构
- 设计审查
 - 检查设计结果
 - 记录发现的设计缺陷（类型、来源、严重性）
- 编写代码
 - 按照编码规范进行代码编写
 - 所编写代码应该是易验证的



软件编码的工作

- 代码走查
 - 确认所写代码完成了所要求的工作
 - 发现的代码缺陷（类型、来源、严重性）
- 编译代码
 - 修改代码的语法错误
- 测试所写代码
 - 对代码进行单元测试
 - 调试代码修改错误

程序员应具备的能力

- **基础知识**
 - 英语的功底
 - 数学基础（算法分析与设计）
 - 计算机科学基础知识
- **基本技能**
 - 认识事务的能力（抽象、模型、结构、层次）
 - 做事的逻辑性和条理性
 - 沟通技能、演讲技巧和团队协作能力
 - 学习新知识和新技术的能力
- **职业化训练和实践经验**

本章要点

- 一、关于编码的一些理念
- 二、编程语言的选择
- 三、编码标准和规范
- 四、程序效率

问题

- 在哪个阶段选择编程语言？
- 是需求阶段确定还是在设计阶段确定，主要看客户有无具体编程环境要求。
- 如果客户提出：在Linux和Windows系统上都能运行，则不能VC++，VB.net等语言。若采用Linux做服务器，那就不能采用ASP语言作为后台程序的开发语言。可选Java和PHP。
- 我们要对程序语言本身的特点进行了解

编程语言的选择

程序语言的分类

- 从计算机发展角度：分为 4 代
 - 1 代：机器语言
 - 2 代：汇编语言
 - 3 代：高级程序设计语言, C, C++, Java
 - 4 代：新一代编程语言, Python

编程语言的选择

程序语言的分类

- 从应用角度：
 - 汇编语言
 - 面向过程的高级语言
 - 面向对象的高级语言
 - 脚本语言
 - ...

编程语言的选择

面向过程的高级语言

- 特点：具有很强的过程功能和数据结构功能，并提供结构化的逻辑构造。
- 代表： PASCAL、PL/1、C

编程语言的选择

面向对象的高级语言

- **Smalltalk** 首先实现真正的面向对象的程序设计，支持程序部件的“可复用性”。
- **C++** 既融合了面向对象的能力，又与C语言兼容，保留了C的许多重要特性。维护了大量已开发的C库、工具及C源程序的完整性。
- 代表：Turbo C++； Borland C++ ； Microsoft C++
- **Java** 是一种简单的面向对象的分布式的语言。功能强大、高效安全，与结构无关，易于移植，是多线程的动态语言。增加了Objective C的扩充，提供更多的动态解决办法。

编程语言的选择

脚本语言：

- 以简单的方式快速完成复杂任务。语法结构简单，使用方便。不需要编译。运行效率略显不足。
- 代表：
 - JavaScript：由Netscape开发，在客户机上执行，专门为制作Web网页而量身定做。
 - PHP：是一种HTML内嵌式语言。是在服务器端执行的嵌入HTML文档的脚本语言。其风格类似于C语言。被许多网站编程人员采用。
 - Perl：用来完成大量不同任务的脚本语言。例如，打印报告，将一个文本文件转换成另一种格式。能在绝大多数操作系统环境下运行。

编程语言的选择

如何选择程序设计语言？ 关系到程序的效率和质量。

应根据软件系统的应用特点，语言的内在特点等选择程序设计语言。

一、语言选择的一般准则

- (1) 项目的应用领域：**应尽量选取适合某个应用领域的语言
- (2) 算法和计算复杂性：**要根据不同语言的特点，选取能够适应软件项目算法和计算复杂性的语言。
- (3) 软件的执行环境：**要选取机器上能运行且具有相应支持软件的语言。

编程语言的选择

(4) 性能因素：应结合工程具体性能来考虑，例如实时系统要求速度，就应选择汇编语言。

(5) 数据结构的复杂性：要根据不同语言构造数据结构类型的能力选取合适的语言。

(6) 软件开发人员的知识水平以及心理因素。

知识水平包括：专业知识，程序设计能力。

心理因素：如对某种语言或工具的熟悉程度。受外界的影响（盲目追求高、新）。

本章要点

- 一、关于编码的一些理念
- 二、编程语言的选择
- 三、编码标准和规范
- 四、程序效率



编码标准和规范

在软件生命期中，程序经常需要被人阅读和理解，如何提高程序的可读性（Readability）？使程序“简单”和“清晰”，进而使程序具有良好的可靠性、可维护性，这是非常重要的。

编码标准和规范

- 标准是建立起来和必须遵守的规则
- 规范是建议最佳做法，推荐更好方式。
- 作为一个开发团队，没有一套规范，大家就会各自为政，为了提高代码质量，不仅需要有很好的程序设计风格，而且需要大家遵守一致的编程规范。
- 例如
 - 注释
 - 变量
 - 格式
 - 文件\目录\约定

编码的风格

- 程序实际上也是一种供人阅读的文章，有一个文章的风格问题。应该使程序具有良好的风格。
- 从20世纪70年代以来，编码的目标从强调效率转变为强调清晰。与此相应，编码的风格从追求“聪明”和“技巧”，变为提倡“简明”和“直接”。
- 人们逐渐认识到，良好的编码风格能在一定程度上弥补语言存在的缺点，反之，不注意风格，即使使用了结构化的现代语言，也很难写出高质量的程序。
- 当多个程序员合作编写一个大的程序时，尤其需要强调良好的和一致的风格，以利于相互通信，减少因不协调而引起的问题。

编码的风格

使用标准的控制结构

源程序文档化

语句结构

数据说明

输入/输出

编码的风格

1、使用标准的控制结构

- 结构化程序设计主要包括两方面：
- 在编写程序时，使用几种基本控制结构，通过组合嵌套，形成程序的控制结构。尽可能避免使用GOTO语句。
- 在程序设计过程中，尽量采用自顶向下和逐步细化的原则，由粗到细，一步步展开。

编码的风格

1、使用标准的控制结构

- 禁止使用GOTO (C 语言) 语句。
- 用IF 语句来强调只执行两组语句中的一组，禁止ELSE GOTO和ELSE RETURN。
- 避免从循环中引出多个出口，应保留函数（方法）只有一个出口。

•问题：以下示例有什么问题？如何修改？

```
p = (char *)malloc(300);  
if (cond1 > 0)  
    strcpy(p, str);  
else return;  
free(p);
```

2、源程序文档化

- 标识符的命名
- 安排注释
- 程序的视觉组织

标识符的命名

- 符号名即标识符，包括模块名、变量名、常量名、数据区名以及缓冲区名等。
- 这些名字应能反映它所代表的实际东西，应有一定实际意义。
- 例如，表示次数的量用 *Times*，表示总量的用 *Total*，表示平均值的用 *Average*，表示和的量用 *Sum* 等。

标识符的命名

- 名字不是越长越好，应当选择精炼的意义明确的名字。必要时可使用缩写名字，但这时要注意缩写规则要一致，并且要给每一个名字加注释。同时，在一个程序中，一个变量只应用于一种用途。
- 例如，在一个程序中定义了一个变量temp，它在程序的前半段代表“Temperature”，在程序的后半段则代表“Temporary”，这使程序阅读者不知所措。

标识符的命名

通用规则：

- 标识符的命名应当直观，可以望文知义。
- 长度符合最小长度下的最大信息。
- 变量名应当使用“名词”或“形容词+名词”
- 函数名应当使用“动词”或者“动词+名词”的形式
- 类和接口名首字母要大写
- 常量名全大写，在单词间用单下划线分隔
- 变量名和参数名第一个单词首字母小写，而后面的单词首字母大写
- ...

标识符的命名

命名规则规范举例

(1) 类名和接口名

```
class CourseOffering ;  
interface Storing;
```

(2) 常量名

```
public static final int MAX_VALUE = 10 ;
```

(3) 全局变量

```
int g_numStudents;
```

(4) 局部变量名

```
float mywidth;
```

一般禁止使用单字符变量名，局部循环可以使用。

比如: `int i, j , k ;`

源程序文档化：程序的注释

2、源程序文档化：程序的注释

- 夹在程序中的注释是程序员与日后的程序读者之间通信的重要手段。
- 注释决不是可有可无的。
- 一些正规的程序文本中，注释行的数量占到整个源程序的 $1/3$ 到 $1/2$ ，甚至更多。
- 注释分为序言性注释和功能性注释。

- 通常置于每个程序模块的开头部分，它应当给出程序的整体说明，对于理解程序本身具有引导作用。有些软件开发部门对序言性注释做了明确而严格的规定，要求程序编制者逐项列出。

```

/*****
** Copyright @ 2003-2008 xxx公司技术开发部
** 创建人： xx
** 日期： xxxxxxxx
** 修改人： xx
** 日期： xxxxxxxx
** 描述：
** 版本：

```


程序的注释 - - 序言性注释

一个描述程序开头的功能及其他程序接口的例子

```
/******  
*****  
  
** 模块功能：寻找两条直线的交点。 **  
  
** 模块名称：FindDPT **  
  
** 代码编写者：张青 **  
  
** 版本：1.1 **  
  
** 日期：2006.10.12 **  
  
** **  
  
** 过程调用：Call FindDPT( A1,B1,C1,A2,B2,C2,XS,YS,Flag) **  
  
** 输入参数：A1,B1,C1,A2,B2,C2 **  
  
**          (直线一：  $A1 \cdot X + B1 \cdot Y + C1 = 0$  **  
  
**          直线二：  $A2 \cdot X + B2 \cdot Y + C2 = 0$ ) **  
  
** 输出参数：如果两条直线平行，Flag=1,否则Flag=0并且 **  
  
**          两条直线的交点是 ( x s , y s )
```

程序的注释 - - 序言性注释

程序的注释 - - 功能性注释

- 功能性注释嵌在源程序体中，用以描述其后的语句或程序段是在做什么工作，或是执行了下面的语句会怎么样。而不要解释下面怎么做。

- 例如

```
/* ADD AMOUNT TO TOTAL */  
TOTAL = AMOUNT + TOTAL
```

不好

功能性注释

功能性注释

- 如果注明把月销售额计入年度总额，便使读者理解了下面语句的意图：

```
/* ADD MONTHLY-SALES TO ANNUAL-TOTAL */
```

```
TOTAL = AMOUNT + TOTAL
```

- 要点

- 描述一段程序，而不是每一个语句；
- 用缩进和空行，使程序与注释容易区别；
- 注释要正确。

程序的视觉组织

程序的视觉组织

- 恰当地利用空格，可以突出运算的优先性，避免发生运算的错误。
- 例如，将表达式
 $(A < -17) \text{ AND NOT } (B < = 49) \text{ OR } C$
写成
 $(A < -17) \text{ AND } \text{ NOT } (B < = 49) \text{ OR } C$
- 自然的程序段之间可用空行隔开
- 可参见相关语言编码规范

程序的视觉组织

程序的视觉组织

- **移行**也叫做**向右缩格**。它是指程序中的各行不必都在左端对齐，都从第一格起排列。这样做使程序完全分不清层次关系。
- 对于**选择语句**和**循环语句**，把其中的程序段语句向右做**阶梯式移行**。使程序的逻辑结构更加清晰。
- 例如，两重选择结构嵌套，写成下面的移行形式，层次就清楚得多。

程序的视觉组织

```
IF (...) THEN  
    IF (...) THEN  
        .....  
    ELSE  
        .....  
    ENDIF  
.....  
ELSE  
    .....  
ENDIF
```

数据说明

★ 3、数据说明

- 在设计阶段已经确定了数据结构的组织及其复杂性。在编写程序时，则需要注意数据说明的风格。
- 为了使程序中数据说明更易于理解和维护，必须注意以下几点。
 1. 数据说明的次序应当规范化
 2. 说明语句中变量安排有序化
 3. 使用注释说明复杂数据结构

数据说明

1、数据说明的次序应当规范化

- 数据说明次序规范化，使数据属性容易查找，也有利于测试，排错和维护。
- **原则上，数据说明的次序与语法无关**，其次序是任意的。但出于阅读、理解和维护的需要，最好使其规范化，使说明的先后次序固定。

数据说明

例如，数据说明次序

- ① 常量说明
- ② 简单变量类型说明
- ③ 数组说明
- ④ 公用数据块说明
- ⑤ 所有的文件说明

在类型说明中还可进一步要求。例如，可按如下顺序排列：

- ① 整型量说明
- ② 实型量说明
- ③ 字符量说明
- ④ 逻辑量说明

数据说明

2、说明语句中变量安排有序化

当多个变量名在一个说明语句中说明时，应当对这些变量按字母的顺序排列。带标号的全程数据(如FORTRAN的公用块)也应当按字母的顺序排列。

例如，把

*integer size, length, width, cost,
price*

写成

*integer cost, length, price , size,
width*

数据说明

3、使用注释说明复杂数据结构

如果设计了一个**复杂的数据结构**，应当使用**注释**来说明在程序实现时这个数据结构的固有特点。

例如，对 $PL/1$ 的链表结构和 $Pascal$ 中用户**自定义的数据类型**，都应当在注释中做必要的补充说明。

★ 4、语句结构

在设计阶段确定了软件的逻辑流结构，但构造单个语句则是编码阶段的任务。语句构造力求简单，直接，不能为了片面追求效率而使语句复杂化。

在一行内只写一条语句

1. 在一行内只写一条语句

在一行内只写一条语句，并且采取适当的移行格式，使程序的逻辑和功能变得更加明确。

许多程序设计语言允许在一行内写多个语句。但这种方式会使程序可读性变差。因而不可取。

在一行内只写一条语句

例如，有一段排序程序

```
FOR I:=1 TO N - 1 DO BEGIN T:=I; FOR J:=I + 1  
TO N DO IF A[J] < A[T] THEN T:=J; IF T≠I  
THEN BEGIN WORK:=A[T]; A[T]:=A[I];  
A[I]:=WORK; END ;END;
```

由于一行中包括了多个语句，掩盖了程序的循环结构和条件结构，使其可读性变得很差。

改进布局

```
FOR I:=1 TO N-1 DO          //改进布局
  BEGIN
    T:=I;
    FOR J:=I+1 TO N DO
      IF A[J] < A[T] THEN T:=J;
    IF T≠I THEN
      BEGIN
        WORK:=A[T];
        A[T]:=A[I];
        A[I]:=WORK;
      END
    END;
  END;
```

程序编写首先应当考虑清晰性

2. 程序编写首先应当考虑清晰性

程序编写首先应当考虑清晰性，不要刻意追求技巧性，使程序编写得过于紧凑。

例如，有一个用 C 语句写出的程序段：

```
A[I] = A[I] + A[T];
```

```
A[T] = A[I] - A[T];
```

```
A[I] = A[I] - A[T];
```


程序编写首先应当考虑清晰性

此段程序可能不易看懂，有时还需用实际数据试验一下。
实际上，这段程序的功能就是交换A[I]和A[T]中的内容。
目的是为了节省一个工作单元。如果改一下：

```
WORK = A[T];
```

```
A[T] = A[I];
```

```
A[I] = WORK;
```

就能让读者一目了然了。

程序要能直截了当地说明程序员的用意

3. 程序要能直截了当地说明程序员的用意。

程序编写得要简单，写清楚，直截了当地说明程序员的用意。例如，

```
for ( i = 1; i <= n; i++ )  
  for ( j = 1; j <= n; j++ )  
    V[i][j] = ( i / j ) * ( j / i )
```

除法运算（/）在除数和被除数都是整型量时，其结果只取整数部分，而得到整型量。

程序要能直截了当地说明程序员的用意

当 $i < j$ 时, $i / j = 0$

当 $j < i$ 时, $j / i = 0$

得到的数组

当 $i \neq j$ 时

$$v[i][j] = (i / j) * (j / i) = 0$$

当 $i = j$ 时

$$v[i][j] = (i / j) * (j / i) = 1$$

这样得到的结果 v 是一个单位矩阵。

程序要能直截了当地说明程序员的用意

写成以下的形式，就能让读者直接了解程序编写者的意图。

```
for ( i = 1; i <= n; i++ )  
    for ( j = 1; j <= n; j++ )  
        if ( i == j )  
            v[i][j] = 1.0;  
        ELSE  
            v[i][j] = 0.0;
```



程序要能直截了当地说明程序员的用意

4. 除非对效率有特殊的要求， 程序编写要做到**清晰第一， 效率第二**。不要为了追求效率而丧失了清晰性。事实上， **程序效率的提高主要通过选择高效的算法来实现**。

5. **首先要保证程序正确， 然后才要求提高速度**。反过来说， 在使程序高速运行时， 首先要保证它是正确的。

程序要能直截了当地说明程序员的用意

6. 避免使用临时变量而使可读性下降。例如，有的程序员为了追求效率，往往喜欢把表达式

$$A[I] + 1 / A[I];$$

写成 $AI = A[I];$

$$X = AI + 1 / AI;$$

这样将一句分成两句写，会产生意想不到的问题。



程序要能直截了当地说明程序员的用意

- 7. 让编译程序做简单的优化。
- 8. 尽可能使用库函数
- 9. 避免不必要的转移。不用 GO TO语句。
- 10. 尽量只采用三种基本的控制结构来编写程序。

程序要能直截了当地说明程序员的用意

11. 避免使用空的ELSE语句和IF... THEN IF...的语句。这种结构容易使读者产生误解。例如，

```
if ( char >= 'a' )  
if ( char <= 'z' )  
cout << "This is a letter."  
else  
cout << "This is not a letter.";
```

else和哪个if对应？可能产生二义性问题。

程序要能直截了当地说明程序员的本意

12. 避免采用过于复杂的条件测试。

13. 尽量减少使用“否定”条件的条件语句。

例如，如果在程序中出现

```
if ( !( char < '0' || char > '9' ) )  
.....
```

改成

```
if ( char >= '0' && char <= '9' )  
.....
```

不要让读者绕弯子想。

程序要能直截了当地说明程序员的用意

- 14. 尽可能用**通俗易懂**的伪码来描述程序的流程，然后再翻译成必须使用的语言。
- 15. 数据结构要有利于程序的简化。
- 16. 要**模块化**，使模块功能尽可能单一化，模块间的耦合能够清晰可见。
- 17. 利用**信息隐蔽**，确保每一个模块的独立性。



程序要能直截了当地说明程序员的用意

- 18. 从**数据**出发去构造程序。
- 19. 不要修补不好的程序，要重新编写。也不要一味地追求代码的复用，要重新组织。
- 20. 对太大的程序，要分块编写、测试，然后再集成。
- 21. **对递归定义的数据结构尽量使用递归过程。**

输入和输出

★ 5、输入和输出

输入和输出信息是与用户的使用直接相关的。输入和输出的方式和格式应当尽可能方便用户的使用。一定要避免因设计不当给用户带来的麻烦。

因此，在软件需求分析阶段和设计阶段，就应基本确定输入和输出的风格。系统能否被用户接受，有时就取决于输入和输出的风格。

输入和输出

不论是批处理的输入 / 输出方式，还是交互式的输入 / 输出方式，在设计和编码时都应考虑下列原则：

1. 对所有的输入数据都要进行检验，识别错误的输入，以保证每个数据的有效性；
2. 检查输入项的各种重要组合的合理性，必要时报告输入状态信息；
3. 使得输入的步骤和操作尽可能简单，并保持简单的输入格式；

输入和输出

4. 输入数据时，应允许使用自由格式输入；
5. 应允许缺省值；
6. 输入一批数据时，最好使用输入结束标志，而不要由用户指定输入数据数目；
7. 在交互式输入输出时，要在屏幕上使用提示符明确提示交互输入的请求，指明可使用选择项的种类和取值范围。同时，在数据输入的过程中和输入结束时，也要在屏幕上给出状态信息；

输入和输出

8. 当程序设计语言对输入 / 输出格式有严格要求时，应保持输入格式与输入语句的要求的**一致性**；
9. 给所有的输出加注解，并设计输出报表格式。

输入 / 输出风格还受到许多其它因素的影响。如输入 / 输出设备（例如终端的类型，图形设备，数字化转换设备等）、用户的熟练程度、以及通信环境等。

本章要点

- 一、关于编码的一些理念
- 二、编程语言的选择
- 三、编码标准和规范
- 四、程序效率

程序效率

★程序效率

讨论效率的准则(时间和空间)

- 程序的效率是指程序的执行速度及程序所需占用的内存的存储空间。
- 程序编码是最后提高运行速度和节省存储的机会，因此在此阶段考虑程序的效率。让我们首先明确讨论程序效率的几条准则

程序效率

效率是一个性能要求，应当在需求分析阶段给出。

软件效率以需求为准，不应以人力所及为准。

- 好的设计可以提高效率。
- 程序的效率与程序的简单性相关。
- 一般说来，任何对效率无重要改善，且对程序的简单性、可读性和正确性不利的程序设计方法都是不可取的。

算法对效率的影响

1、算法对效率的影响

- 源程序的**效率与详细设计阶段确定的算法的效率直接有关。**
- 在详细设计翻译转换成源程序代码后，**算法效率反映为程序的执行速度和存储容量的要求。**

算法对效率的影响

设计向程序转换过程中的指导原则：

- ① 在编程序前，尽可能化简有关的算术表达式和逻辑表达式；
- ② 仔细检查算法中的嵌套的循环，尽可能将某些语句或表达式移到循环外面；
- ③ 尽量避免使用多维数组；
- ④ 尽量避免使用指针和复杂的表；
- ⑤ 采用“快速”的算术运算；

算法对效率的影响

- ⑥ 不要混淆数据类型，避免在表达式中出现类型混杂；
- ⑦ 尽量采用整数算术表达式和布尔表达式；
- ⑧ 选用等效的高效率算法；
- ⑨ 许多编译程序具有“优化”功能，可以自动生成高效率的目标代码。

影响存储器效率的因素

2、影响存储器效率的因素

在大中型计算机系统中，存储限制不再是主要问题。在这种环境下，对内存采取基于操作系统的分页功能的虚拟存储管理。存储效率与操作系统的分页功能直接有关。

影响存储器效率的因素

- 采用结构化程序设计，将程序功能合理分块，使每个模块或一组密切相关模块的程序体积大小与每页的容量相匹配，可减少页面调度，减少内外存交换，提高存储效率。
- 在微型计算机系统中，存储器的容量对软件设计和编码的制约很大。因此要选择可生成较短目标代码且存储压缩性能优良的编译程序，有时需采用汇编程序。
- 提高存储器效率的关键是程序的简单性。

影响输入 / 输出的因素

3、影响输入 / 输出的因素

输入 / 输出可分为两种类型：

面向人(操作员)的输入 / 输出

面向设备的输入 / 输出

如果操作员能够十分方便、简单地录入输入数据，或者能够十分直观、一目了然地了解输出信息，则可以说面向人的输入 / 输出是高效的。

影响输入 / 输出的因素

- 关于面向设备的输入/输出，可以提出一些提高输入/输出效率的指导原则：
 - 输入/输出的请求应当最小化；
 - 对于所有的输入/输出操作，安排适当的缓冲区，以减少频繁的信息交换。
 - 对辅助存储（例如磁盘），选择尽可能简单的，可接受的存取方法；